OpenNotation

Master Architecture & Implementation Plan

Cursor.AI Obsessively Detailed Build Guide

Core Principle

Notation edits the score.
Engraving computes the page.
Rendering draws the result.

Version 1.0 — TypeScript + Rust — Leland OTF (SMuFL)

Table of Contents

Chapter 0 — System Philosophy & Architecture Overview

OpenNotation is not a notation app. It is a compiler pipeline for musical semantics. Understanding this distinction is the single most important prerequisite before writing any code.

0.1 The Five Conceptual Layers

Every piece of data, every function, and every module must live in exactly one of the following five layers. If you cannot decide which layer something belongs to, the architecture is being violated.

Layer	Responsibility
`Semantic Layer`	Musical meaning: notes, durations, voices, measures, articulations. This is the source of truth. It has NO coordinates.
`Engraving Layer`	Deterministic layout compiler. Consumes the semantic model, outputs a geometry tree: positions, spacings, skylines, beam paths.
`Render Layer`	GPU-backed drawing system. Consumes geometry. Outputs pixels. Knows nothing about music.
`Interaction Layer`	State machine: input events → notation commands → score mutations → engraving reflow → render update.
`Platform Layer`	Tauri / Electron / React / Web adapter. Abstracts all platform-specific concerns. The engine core is platform-agnostic.

0.2 The Compiler Pipeline

The internal data flow is unidirectional and deterministic. Think of it exactly like a compiler:

```
Music AST (Score Model)
    │
    ▼
Notation Mutation Layer     ← All edits happen here, via Command objects
    │
    ▼
Engraving Compiler          ← Deterministic layout: spacing, beams, collision
    │
    ▼
Scene Graph                 ← Pure geometry: positions, glyphs, paths
    │
    ▼
GPU Renderer                ← WebGPU / Canvas 2D draws the scene graph
```

> **Critical Rule**
> NEVER store absolute pixel coordinates in the score model.

> NEVER let the renderer know what a "note" is.
> NEVER let the semantic layer know what a "pixel" is.
> Layout must always be recomputable from the semantic model alone.

0.3 The Layout Position Formula

Every rendered object's final position follows exactly one formula, borrowed from MuseScore's architecture:

```
  Display Position  =  AP  +  LO  +  UO

  AP  = Anchor Position     (computed from staff geometry and tick position)
  LO  = Layout Offset       (computed by the engraving engine: collision, spacing)
  UO  = User Offset         (stored in the score model: user has dragged this
object)
```

This formula is sacred. The user can drag a dynamic marking up. That UO is stored in the score model. When the score re-engravs, the engraving engine re-computes AP+LO from scratch, then adds UO on top. User customizations are never destroyed by re-layout.

0.4 What OpenNotation Is NOT

- A canvas that lets you draw notes freely.
- A WYSIWYG pixel editor.
- A DOM-based SVG renderer.
- An opinionated UI framework.
- Tied to any specific playback engine (Tone.js is a plugin, not a requirement).
- Tied to any specific music font (Leland OTF is the default, but it is swappable).

0.5 Technology Stack Decisions

Component	Decision & Rationale
`Core Language`	TypeScript. All public APIs, the score model, the engraving engine, the interaction system.
`Performance Core`	Optional Rust compiled to WASM for hot paths: spacing solver, collision engine, playback traversal.
`GPU Rendering`	WebGPU primary target. Canvas 2D as software fallback.
`Music Font`	Leland OTF from MuseScore — open-source, SMuFL-compliant. All glyphs are accessed by SMuFL codepoint.
`Playback`	Adapter-based. Default adapter wraps Tone.js. Custom adapters implement the IPlaybackAdapter interface.

Component	Decision & Rationale
`Serialization`	MusicXML (interop), JSON (native), MIDI (export), compressed binary cache for large scores.
`Desktop`	Tauri (Rust backend) or Electron (Node.js backend). The engine runs identically in both.
`Web`	React bindings provided. But engine core has zero React dependency.
`Build`	Vite or ESBuild for library bundling. Vitest for unit tests.

Phase 0 — Project Setup & Repository Architecture

This phase produces zero musical functionality. Its only job is to establish the repository structure, build toolchain, TypeScript configuration, module boundaries, and testing harness that every subsequent phase depends on. Do not rush this phase. An incorrect structure here forces painful refactoring later.

0.A Repository Structure

The repository is a monorepo. Each directory under src/core/ is a self-contained module with its own index.ts barrel export and zero circular dependencies. Enforce this with ESLint import rules.

```
opennotation/
├── src/
│   ├── core/
│   │   ├── model/           ← Phase 1: Score AST, all musical data types
│   │   ├── temporal/        ← Phase 2: Tick system, slices, timing
│   │   ├── engraving/       ← Phase 3: Engraving compiler
│   │   │   ├── spacing/     ← Rhythmic spacing solver
│   │   │   ├── beaming/     ← Beam group computation
│   │   │   ├── collision/   ← Skyline collision system
│   │   │   ├── stems/       ← Stem direction and length
│   │   │   ├── accidentals/← Accidental placement
│   │   │   ├── slurs/       ← Slur and tie layout
│   │   │   └── systems/     ← System and page breaking
│   │   ├── rendering/       ← Phase 5: Scene graph + GPU renderer
│   │   │   ├── scenegraph/ ← Render tree data structures
│   │   │   ├── webgpu/      ← WebGPU backend
│   │   │   ├── canvas2d/    ← Software fallback renderer
│   │   │   └── cache/       ← Page and glyph cache
│   │   ├── editor/          ← Phase 6: Interaction state machine
│   │   │   ├── commands/    ← Command objects (undo/redo)
│   │   │   ├── input/       ← Input pipeline
│   │   │   ├── selection/   ← Selection model
│   │   │   └── noteentry/   ← Note entry mode
│   │   ├── playback/        ← Phase 7: Playback traversal engine
│   │   │   ├── traversal/   ← Measure/beat/event walker
│   │   │   └── adapters/    ← IPlaybackAdapter + Tone.js adapter
│   │   ├── serialization/   ← Phase 8: MusicXML, JSON, MIDI
│   │   │   ├── musicxml/    ← MusicXML import/export
│   │   │   ├── json/        ← Native JSON format
│   │   │   └── midi/        ← MIDI export
│   │   ├── fonts/           ← Phase 4: Leland OTF loader + SMuFL map
│   │   └── wasm/            ← Phase 10: WASM bridge for Rust modules
│   │
│   ├── platform/            ← Platform adapters
│   │   ├── web/             ← Browser-specific glue
│   │   ├── tauri/           ← Tauri IPC adapter
│   │   └── electron/        ← Electron IPC adapter
│   │
│   └── bindings/            ← Framework bindings (no engine logic here)
│       ├── react/           ← React hooks and components
│       └── vanilla/         ← Vanilla JS API surface
│
├── rust/                    ← Rust WASM modules (optional, Phase 10)
│   ├── spacing/             ← Spring-based spacing solver in Rust
```

```
│   └── collision/          ← Skyline algorithm in Rust
│
├── fonts/                  ← Leland.otf + metadata.json (SMuFL glyphnames)
├── tests/                  ← Integration tests mirroring src/core/ structure
├── examples/               ← Example scores (MusicXML + JSON)
├── docs/                   ← API documentation
├── package.json
├── tsconfig.json
├── vite.config.ts
└── vitest.config.ts
```

0.B TypeScript Configuration

The tsconfig.json must enforce strict type checking. Every interface must be explicit. Implicit any is a bug, not a shortcut.

```
// tsconfig.json
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "strict": true,
    "exactOptionalPropertyTypes": true,
    "noUncheckedIndexedAccess": true,
    "noImplicitOverride": true,
    "paths": {
      "@opennotation/model":       ["./src/core/model/index.ts"],
      "@opennotation/temporal":    ["./src/core/temporal/index.ts"],
      "@opennotation/engraving":   ["./src/core/engraving/index.ts"],
      "@opennotation/rendering":   ["./src/core/rendering/index.ts"],
      "@opennotation/editor":      ["./src/core/editor/index.ts"],
      "@opennotation/playback":    ["./src/core/playback/index.ts"],
      "@opennotation/fonts":       ["./src/core/fonts/index.ts"],
      "@opennotation/serial":      ["./src/core/serialization/index.ts"]
    }
  }
}
```

0.C Module Boundary Rules

These rules are ENFORCED by ESLint. Cursor.AI must never violate them.

- model/ has ZERO imports from any other src/core module.
- temporal/ may import from model/ only.
- engraving/ may import from model/ and temporal/ only.
- rendering/ may import from engraving/ only. It never imports from model/.
- editor/ may import from model/, temporal/, engraving/. Never from rendering/.
- playback/ may import from model/ and temporal/ only.
- serialization/ may import from model/ only.
- fonts/ has ZERO imports from any other module.
- Platform adapters may import from any core module.

- Framework bindings may import from any module.

> **Why This Matters**
> These import boundaries are what allow you to run the engraving engine in a WASM context without pulling in the DOM.
> They are what allow you to use the score model in a Node.js serialization script without loading WebGPU.
> Circular imports WILL cause subtle initialization bugs that are nearly impossible to trace. Do not allow them.

0.D Package.json & Build Setup

```
{
  "name": "@opennotation/core",
  "version": "0.1.0",
  "type": "module",
  "exports": {
    ".":              "./dist/index.js",
    "./model":        "./dist/core/model/index.js",
    "./engraving":    "./dist/core/engraving/index.js",
    "./rendering":    "./dist/core/rendering/index.js",
    "./editor":       "./dist/core/editor/index.js",
    "./playback":     "./dist/core/playback/index.js",
    "./react":        "./dist/bindings/react/index.js"
  },
  "scripts": {
    "build":   "vite build",
    "test":    "vitest run",
    "test:watch": "vitest",
    "typecheck": "tsc --noEmit"
  }
}
```

0.E Initial Files to Create in This Phase

Create these files as empty shells with correct imports and placeholder exports. Do NOT implement them yet. The goal is a buildable, type-checkable, test-runnable project skeleton.

1. src/core/model/index.ts — export * from each model file
2. src/core/temporal/index.ts
3. src/core/engraving/index.ts
4. src/core/rendering/index.ts
5. src/core/editor/index.ts
6. src/core/playback/index.ts
7. src/core/fonts/index.ts
8. src/core/serialization/index.ts
9. src/platform/web/index.ts
10. src/bindings/react/index.ts

11. tests/smoke.test.ts — import each module, assert it exports a truthy value

0.F Deliverable for Phase 0

- Repository compiles with tsc --noEmit with zero errors.
- vitest smoke test passes.
- ESLint import rules are configured and passing.
- All module directories exist with index.ts barrel files.
- fonts/ directory contains Leland.otf and smufl-metadata.json.

Phase 1 — Semantic Score Model

The score model is the source of truth for the entire system. Everything else is derived from it. Build this with obsessive care. It cannot be partially correct — any ambiguity here propagates as bugs through engraving, playback, and serialization.

The model is a purely semantic data structure. It contains musical meaning. It never contains pixel coordinates, screen positions, rendered glyph references, or any rendering concern.

1.1 Core Concepts

Ticks

Time in the score is measured in ticks — an integer unit of musical time. The default tick resolution is 480 ticks per quarter note (TPQ). This matches the MIDI standard and makes common subdivisions exact integers:

```
const TPQ = 480; // Ticks Per Quarter note

// Duration in ticks:
whole        = TPQ * 4   = 1920
half         = TPQ * 2   = 960
quarter      = TPQ       = 480
eighth       = TPQ / 2   = 240
sixteenth    = TPQ / 4   = 120
32nd         = TPQ / 8   = 60
dotted quarter = TPQ * 3/2 = 720
triplet quarter = TPQ * 2/3 = 320
```

Voices

Each staff has up to 4 independent voices. Voice 1 stems go up, voice 2 stems go down. Voices 3 and 4 follow the same convention. Each voice is an independent sequence of ChordEvents and RestEvents.

Spanners

Spanners are musical elements that connect two points in the score: slurs, ties, hairpins, ottava lines, voltas, trills. They are stored separately from note events and reference their start and end points by tick + staff + voice + element ID.

1.2 The Complete Type System

Every type in the following section lives in src/core/model/. File names are given for each type group.

File: src/core/model/primitives.ts

```
  // The fundamental musical primitives.
  // These are pure data — no methods, no class instances.

  export type Pitch = {
    step:   "C" | "D" | "E" | "F" | "G" | "A" | "B";
    octave: number;       // 0–9, middle C is octave 4
    alter:  number;       // -2 = double flat, -1 = flat, 0 = natural, 1 = sharp, 2
= double sharp
  };

  export type Duration = {
    type:   NoteType;     // see NoteType enum below
    dots:   number;       // 0 = undotted, 1 = dotted, 2 = double-dotted
    ticks:  number;       // ALWAYS computed: do not store manually
  };

  export type NoteType =
    | "maxima" | "long" | "breve"
    | "whole" | "half" | "quarter"
    | "eighth" | "16th" | "32nd" | "64th" | "128th";

  export type Accidental =
    | "sharp" | "flat" | "natural"
    | "double-sharp" | "double-flat"
    | "sharp-up" | "flat-down"     // microtonal
    | null;                         // no visible accidental (courtesy)

  export type ClefType =
    | "treble" | "bass" | "alto" | "tenor"
    | "treble8vb" | "bass8vb" | "percussion";

  export type KeySignature = {
    fifths: number;   // -7 to +7: -1=F major/D minor, +1=G major/E minor, etc.
    mode:   "major" | "minor";
  };

  export type TimeSignature = {
    numerator:   number;
    denominator: number;   // must be a power of 2
    symbol?: "common" | "cut"; // override display as C or alla breve
  };
```

File: src/core/model/events.ts

A ScoreEvent is the atomic unit of musical content in the score. Every event has a unique ID, a tick position, and belongs to exactly one voice on one staff.

```
  import type { Pitch, Duration, Accidental, ClefType,
                KeySignature, TimeSignature } from "./primitives";

  // Every event in the score inherits from this base.
  export type ScoreEventBase = {
    id:      string;   // UUID, immutable after creation
    tick:    number;   // absolute position from score start
    staffId: string;
    voiceId: number;   // 1–4
  };

  // A single note (or grace note) with optional articulations.
  export type NoteEvent = ScoreEventBase & {
```

```
  kind:          "note";
  pitch:         Pitch;
  duration:      Duration;
  accidental:    Accidental;
  tieStart:      boolean;   // this note starts a tie
  tieEnd:        boolean;   // this note ends a tie
  grace:         boolean;
  graceSlash:    boolean;   // acciaccatura slash
  articulations: ArticulationType[];
  dynamic?:      DynamicType;
  fingering?:    string;
  technicals:    TechnicalType[];
  lyric?:        LyricEvent;
};

// A chord is multiple notes at the same tick/voice/staff.
// Internally, the first NoteEvent is the "primary" note,
// and additional notes in the chord reference its ID.
export type ChordMember = {
  noteId:  string;   // references a NoteEvent.id
  chordId: string;   // shared ID for all members of this chord
};

export type RestEvent = ScoreEventBase & {
  kind:     "rest";
  duration: Duration;
  hidden?:  boolean;   // invisible rest (spacer)
};

export type ClefEvent = ScoreEventBase & {
  kind: "clef";
  clef: ClefType;
  line: number;   // staff line (default per clef type)
};

export type KeySigEvent = ScoreEventBase & {
  kind:   "keysig";
  key:    KeySignature;
  cancel: boolean;   // show cancellation naturals
};

export type TimeSigEvent = ScoreEventBase & {
  kind: "timesig";
  time: TimeSignature;
};

export type TempoEvent = ScoreEventBase & {
  kind:        "tempo";
  bpm:         number;
  beatUnit:    NoteType;   // what note value = bpm
  text?:       string;     // "Allegro", "Andante", etc.
  fermata?:    boolean;
};

export type DynamicEvent = ScoreEventBase & {
  kind:    "dynamic";
  dynamic: DynamicType;
  userOffset: { x: number; y: number }; // UO in layout formula
};

export type TextEvent = ScoreEventBase & {
  kind:       "text";
```

```
  text:       string;
  style:      TextStyle;
  userOffset: { x: number; y: number };
};

export type BarlineEvent = ScoreEventBase & {
  kind:    "barline";
  barline: BarlineType;
};

// Union of all event types
export type ScoreEvent =
  | NoteEvent | RestEvent | ClefEvent | KeySigEvent
  | TimeSigEvent | TempoEvent | DynamicEvent | TextEvent | BarlineEvent;

// Enumerations
export type ArticulationType =
  | "staccato" | "tenuto" | "accent" | "marcato"
  | "staccatissimo" | "portato" | "stress" | "unstress";

export type DynamicType =
  | "pppp" | "ppp" | "pp" | "p" | "mp"
  | "mf" | "f" | "ff" | "fff" | "ffff"
  | "sfz" | "sfp" | "rfz" | "fp";

export type TechnicalType =
  | "up-bow" | "down-bow" | "harmonic" | "open-string"
  | "stopped" | "snap-pizzicato" | "left-hand-pizzicato";

export type BarlineType =
  | "regular" | "dotted" | "dashed" | "heavy"
  | "light-light" | "light-heavy" | "heavy-light"
  | "heavy-heavy" | "short" | "tick"
  | "repeat-start" | "repeat-end" | "repeat-end-start";

export type TextStyle = {
  fontFamily?: string;
  fontSize?:   number;
  bold?:       boolean;
  italic?:     boolean;
  color?:      string;
};

export type NoteType =
  "whole" | "half" | "quarter" | "eighth" | "16th" | "32nd" | "64th" | "128th";

export type LyricEvent = {
  text:     string;
  syllabic: "single" | "begin" | "middle" | "end";
  verse:    number;
};
```

File: src/core/model/spanners.ts

Spanners are score-level objects that span between two events. They are NOT embedded in events — they live at the score level and reference events by ID.

```
export type SpannerBase = {
  id:        string;
  startId:   string;   // ScoreEvent.id where spanner begins
```

```
  endId:     string;   // ScoreEvent.id where spanner ends
  staffId:   string;
  voiceId:   number;
  userOffset: { x: number; y: number };
};

export type SlurSpan = SpannerBase & {
  kind:      "slur";
  placement: "above" | "below";
  bezier?: { x1: number; y1: number; x2: number; y2: number };
};

export type TieSpan = SpannerBase & {
  kind:      "tie";
  placement: "above" | "below";
};

export type HairpinSpan = SpannerBase & {
  kind:   "hairpin";
  shape:  "crescendo" | "decrescendo";
  spread: number; // max opening width in staff spaces
};

export type OttavaSpan = SpannerBase & {
  kind:  "ottava";
  type:  "8va" | "8vb" | "15ma" | "15mb";
};

export type VoltaSpan = SpannerBase & {
  kind:    "volta";
  numbers: number[];  // [1] or [2] or [1,2]
  closed:  boolean;   // volta with closing bracket
};

export type PedalSpan = SpannerBase & {
  kind:  "pedal";
  style: "bracket" | "sign" | "line";
};

export type Spanner =
  | SlurSpan | TieSpan | HairpinSpan | OttavaSpan | VoltaSpan | PedalSpan;
```

File: src/core/model/structure.ts

Structural elements organize the score into measures, staves, parts, and voices.

```
export type Measure = {
  id:        string;
  number:    number;       // 1-based display number
  tick:      number;       // absolute start tick
  duration:  number;       // duration in ticks (matches time signature)
  timeSig?:  TimeSignature; // only present if changed from previous measure
  keySig?:   KeySignature;  // only present if changed
  clefs:     Record<string, ClefType>; // staffId → clef (only if changed)
  repeatStart: boolean;
  repeatEnd:   boolean;
  repeatCount: number;   // how many times to repeat (default 2)
  sectionBreak?: boolean;
  pageBreak?:    boolean;
  systemBreak?:  boolean;
```

```
  rehearsalMark?: string;
};

export type Staff = {
  id:          string;
  partId:      string;
  index:       number;  // 0-based within part (0=top, 1=bottom for piano)
  lineCount:   number;  // 5 for standard, 1 for percussion, 6 for guitar tab
  distance:    number;  // staff distance from previous staff in staff spaces
  showBraces:  boolean;
  defaultClef: ClefType;
};

export type Part = {
  id:          string;
  name:        string;         // full name: "Violin I"
  abbreviation: string;        // short name: "Vln. I"
  staffIds:    string[];       // ordered list of staff IDs
  instrument:  Instrument;
  midiChannel: number;         // 0-15
  midiProgram: number;         // 0-127 General MIDI
};

export type Instrument = {
  id:          string;
  name:        string;
  family:      string;
  transposition?: {
    chromatic:   number;  // semitones
    diatonic:    number;  // scale steps
  };
};

export type TupletGroup = {
  id:          string;
  staffId:     string;
  voiceId:     number;
  startTick:   number;
  endTick:     number;
  actual:      number;  // notes in the tuplet (e.g., 3)
  normal:      number;  // notes in the normal space (e.g., 2)
  showBracket: boolean;
  showNumber:  "actual" | "both" | "none";
  eventIds:    string[];
};

export type BeamGroup = {
  id:       string;
  eventIds: string[];  // ordered NoteEvent IDs in this beam group
};
```

File: src/core/model/score.ts — The Root

```
// The Score is the root of the entire musical data structure.
// It is the ONLY object passed between layers.
export type Score = {
  id:       string;
  metadata: ScoreMetadata;

  // Structure
```

```
  parts:    Part[];
  staves:   Staff[];
  measures: Measure[];

  // Events — flat map for O(1) lookup by ID
  events:   Map<string, ScoreEvent>;

  // Events ordered by tick for each voice
  // Key: `${staffId}:${voiceId}`
  eventIndex: Map<string, string[]>;  // → ordered event IDs

  // Spanners
  spanners: Map<string, Spanner>;

  // Beam groups
  beamGroups: Map<string, BeamGroup>;

  // Tuplet groups
  tuplets: Map<string, TupletGroup>;

  // The TimeMap: maps absolute ticks to musical positions
  timeMap: TimeMap;
};

export type ScoreMetadata = {
  title:         string;
  subtitle?:     string;
  composer?:     string;
  lyricist?:     string;
  arranger?:     string;
  copyright?:    string;
  movementTitle?: string;
  movementNumber?: string;
  createdDate:   string;   // ISO 8601
  modifiedDate:  string;
  encodingDescription?: string;
};

// The TimeMap resolves tick → musical position for any point in the score.
// Accounts for tempo changes, repeats, and anacrusis.
export type TimeMap = {
  entries: TimeMapEntry[];
};

export type TimeMapEntry = {
  tick:         number;
  measureNumber: number;
  beatIndex:    number;  // 0-based beat within measure
  bpm:          number;  // tempo at this tick
  realTimeMs:   number;  // milliseconds from score start (no repeats)
};
```

1.3 Score Factory Functions

Never allow direct object literals to construct Score nodes. Provide factory functions that enforce invariants and generate IDs.

```
// File: src/core/model/factory.ts
```

```
  export function createScore(metadata: Partial<ScoreMetadata>): Score { ... }

  export function createPart(opts: {
    name: string;
    abbreviation: string;
    staffCount: number;
    instrument: Instrument;
  }): { part: Part; staves: Staff[] } { ... }

  export function createMeasure(opts: {
    number: number;
    tick: number;
    timeSig: TimeSignature;
  }): Measure { ... }

  export function createNoteEvent(opts: {
    staffId: string;
    voiceId: number;
    tick: number;
    pitch: Pitch;
    duration: Duration;
  }): NoteEvent { ... }

  export function createRestEvent(opts: {
    staffId: string;
    voiceId: number;
    tick: number;
    duration: Duration;
  }): RestEvent { ... }

  // Utility: compute tick duration from NoteType + dots
  export function computeTicks(type: NoteType, dots: number, tpq = 480): number {
... }

  // Utility: pitch to MIDI note number
  export function pitchToMidi(pitch: Pitch): number { ... }

  // Utility: MIDI note number to pitch
  export function midiToPitch(midi: number): Pitch { ... }

  // Utility: pitch to staff line/space position
  // Returns a number where 0 = bottom line, 0.5 = first space, 1 = second line,
etc.
  export function pitchToStaffPosition(pitch: Pitch, clef: ClefType): number { ...
}
```

1.4 Score Query Functions

Provide a rich query API so other modules can read the score without coupling to its internal structure.

```
  // File: src/core/model/queries.ts

  // Get all events in a measure for a given staff/voice, ordered by tick.
  export function getEventsInMeasure(
    score: Score, measureId: string, staffId: string, voiceId: number
  ): ScoreEvent[] { ... }

  // Get the clef active at a given tick on a given staff.
```

```
export function getActiveClef(
  score: Score, staffId: string, tick: number
): ClefType { ... }

// Get the key signature active at a given tick.
export function getActiveKeySignature(
  score: Score, tick: number
): KeySignature { ... }

// Get the time signature active at a given tick.
export function getActiveTimeSig(
  score: Score, tick: number
): TimeSignature { ... }

// Get the measure containing a given tick.
export function getMeasureAtTick(
  score: Score, tick: number
): Measure | undefined { ... }

// Get all notes in a chord (same tick, same staff, same voice).
export function getChordNotes(
  score: Score, chordId: string
): NoteEvent[] { ... }

// Get all spanners that touch a given tick range.
export function getSpannersInRange(
  score: Score, staffId: string, startTick: number, endTick: number
): Spanner[] { ... }

// Compute the real-time millisecond position of a tick.
export function tickToMs(
  score: Score, tick: number
): number { ... }
```

1.5 Phase 1 Deliverables

- All types defined with zero TypeScript errors.
- Factory functions implemented and unit-tested.
- Query functions implemented and unit-tested.
- At minimum 40 unit tests covering: pitch conversion, tick computation, chord queries, measure queries, spanner queries.
- A createMinimalScore() helper that builds a score with 1 part, 1 staff, 4/4 time, C major, 4 empty measures — used as the test fixture for all subsequent phases.

Phase 2 — Temporal Engine (Tick System & Slices)

The temporal engine is the backbone of engraving and playback. It normalizes the heterogeneous events of all staves and voices into a single time-ordered structure: the temporal slice map. Every subsequent phase depends on this.

2.1 What Is a Temporal Slice?

A temporal slice is a snapshot of everything happening at a specific tick across ALL staves and ALL voices. It is the foundation of time alignment in notation.

```
// File: src/core/temporal/slice.ts

export type TemporalSlice = {
  tick:      number;
  duration:  number;   // ticks until next slice (minimum event duration at this
tick)
  events:    SliceEvent[];
};

export type SliceEvent = {
  eventId:  string;
  staffId:  string;
  voiceId:  number;
  duration: number;   // duration of this specific event in ticks
};

// The full slice map: ordered array of TemporalSlice from tick 0 onward.
export type TemporalSliceMap = {
  slices:       TemporalSlice[];
  sliceByTick:  Map<number, TemporalSlice>;   // O(1) lookup
  totalTicks:   number;
};
```

2.2 Building the Slice Map

```
// File: src/core/temporal/sliceBuilder.ts

export function buildSliceMap(score: Score): TemporalSliceMap {
  // Step 1: Collect all unique tick positions from ALL events
  //         across ALL staves and ALL voices.
  const tickSet = new Set<number>();
  for (const event of score.events.values()) {
    tickSet.add(event.tick);
    if ("duration" in event) {
      tickSet.add(event.tick + event.duration.ticks);
    }
  }

  // Step 2: Sort ticks.
  const ticks = [...tickSet].sort((a, b) => a - b);

  // Step 3: For each tick, collect all events starting at that tick.
  const slices: TemporalSlice[] = [];
  for (let i = 0; i < ticks.length; i++) {
```

```
      const tick = ticks[i]!;
      const duration = (ticks[i + 1] ?? tick) - tick;
      const events: SliceEvent[] = [];
      for (const event of score.events.values()) {
        if (event.tick === tick) {
          events.push({
            eventId:  event.id,
            staffId:  event.staffId,
            voiceId:  event.voiceId,
            duration: "duration" in event ? event.duration.ticks : 0,
          });
        }
      }
      if (events.length > 0 || i === 0) {
        slices.push({ tick, duration, events });
      }
    }

    // Step 4: Build O(1) lookup map.
    const sliceByTick = new Map<number, TemporalSlice>();
    for (const slice of slices) sliceByTick.set(slice.tick, slice);

    return { slices, sliceByTick, totalTicks: ticks.at(-1) ?? 0 };
  }
```

2.3 Rhythmic Spacing Primitives

The temporal engine also provides the spacing weight for each slice — how much horizontal space a given duration deserves. This is computed here but consumed by the engraving engine.

```
  // File: src/core/temporal/spacing.ts

  // Proportional spacing: the visual width of a duration is proportional to
  // log2 of its tick count. This matches professional engraving practice.
  // Reference: Ross, "The Art of Music Engraving"

  export function durationToSpacingWidth(ticks: number, tpq = 480): number {
    // Base width in staff spaces for a quarter note.
    const BASE_QUARTER = 1.8;
    // Logarithmic proportional width.
    const ratio = ticks / tpq;  // 1.0 = quarter, 2.0 = half, 0.5 = eighth
    return BASE_QUARTER * Math.pow(ratio, 0.55);
    // The 0.55 exponent is a common approximation for music spacing.
    // Perfect linear spacing (1.0) is too wide for long notes.
    // Square root (0.5) compresses short notes too much.
    // 0.55 is empirically correct for most notation.
  }

  // Minimum width for a slice, considering all events it contains.
  // Short notes might need extra space for accidentals or articulations.
  export function computeMinimumSliceWidth(
    slice: TemporalSlice,
    score: Score,
    tpq = 480
  ): number {
    const BASE = 0.5;  // minimum in staff spaces
    let min = BASE;
    const durations = slice.events
      .map(e => { const ev = score.events.get(e.eventId)!;
```

```
                  return "duration" in ev ? ev.duration.ticks : 0; })
      .filter(d => d > 0);
    if (durations.length === 0) return min;
    const shortest = Math.min(...durations);
    return Math.max(min, durationToSpacingWidth(shortest, tpq));
  }
```

2.4 TimeMap Builder

The TimeMap converts between tick positions and real-world milliseconds, accounting for all tempo changes in the score.

```
// File: src/core/temporal/timemap.ts

export function buildTimeMap(score: Score, tpq = 480): TimeMap {
  // Collect all tempo events, sorted by tick.
  const tempoEvents = [...score.events.values()]
    .filter((e): e is TempoEvent => e.kind === "tempo")
    .sort((a, b) => a.tick - b.tick);

  const entries: TimeMapEntry[] = [];
  let currentBpm = 120;
  let currentRealTimeMs = 0;
  let currentTick = 0;

  const msPerTick = (bpm: number) => (60000 / bpm) / tpq;

  // Add the default 120 BPM entry at tick 0 if no tempo event exists there.
  if (!tempoEvents.find(t => t.tick === 0)) {
    tempoEvents.unshift({ kind: "tempo", tick: 0, bpm: 120 } as TempoEvent);
  }

  for (const tempo of tempoEvents) {
    // Advance real time up to this tempo event.
    if (tempo.tick > currentTick) {
      currentRealTimeMs += (tempo.tick - currentTick) * msPerTick(currentBpm);
    }
    currentBpm = tempo.bpm;
    currentTick = tempo.tick;
    const measure = getMeasureAtTick(score, tempo.tick);
    entries.push({
      tick:          tempo.tick,
      measureNumber: measure?.number ?? 1,
      beatIndex:     0,  // simplified; full impl tracks beat subdivisions
      bpm:           currentBpm,
      realTimeMs:    currentRealTimeMs,
    });
  }

  return { entries };
}
```

2.5 Phase 2 Deliverables

- buildSliceMap() implemented and tested with multi-staff, multi-voice scores.
- durationToSpacingWidth() unit tested against known professional spacing ratios.

- buildTimeMap() unit tested: tempo change at measure 5 correctly shifts all subsequent real-time values.
- Test: a score with voices 1 and 2 having different rhythmic values produces slices at every subdivision boundary.

Phase 3 — Engraving Engine

The engraving engine is the heart of OpenNotation. It is a deterministic compiler that takes a Score and a set of layout parameters, and produces a complete geometry tree ready for rendering. It must produce identical output for identical input — always.

This phase is the largest and most complex. It is broken into sub-phases, each building on the previous.

3.1 The Engraving Pipeline Overview

```
  Input:  Score + LayoutParameters
  Output: EngravingResult (geometry tree)

  Pipeline steps (in strict order):

  1.  Build TemporalSliceMap          (from Phase 2)
  2.  Resolve beam groups              (which notes beam together)
  3.  Compute stem directions          (per voice, per beam group)
  4.  Compute stem lengths             (avoid collisions with beams/flags)
  5.  Place accidentals                (left-to-right, collision-aware)
  6.  Compute rhythmic spacing         (spring-based: minimum + proportional)
  7.  Build measure geometry           (note columns, barlines, clef/keysig/timesig)
  8.  Run skyline collision            (avoid dynamics, articulations, lyrics
colliding)
  9.  Break into systems               (line-breaking algorithm)
  10. Justify systems                  (stretch notes to fill system width)
  11. Place page elements              (headers, footers, rehearsal marks)
  12. Compute slur/tie geometry        (Bezier curves, avoiding staff objects)
  13. Output EngravingResult
```

> **Golden Rule**
> The engraving engine is a pure function: (Score, LayoutParams) → EngravingResult.
> It has NO side effects. It does NOT mutate the score. It does NOT touch the DOM.
> This makes it trivially testable, cacheable, and WASM-compilable.

3.2 Engraving Types

```
  // File: src/core/engraving/types.ts

  // The top-level result of the engraving pipeline.
  export type EngravingResult = {
    pages:      EngravingPage[];
    totalPages: number;
  };

  export type EngravingPage = {
    index:    number;
    width:    number;   // in staff spaces (1 staff space = distance between 2
lines)
    height:   number;
```

```
  systems:  EngravingSystem[];
  pageElements: EngravingElement[];  // title, composer, copyright text
};

export type EngravingSystem = {
  id:          string;
  x: number; y: number;   // top-left in page coordinates (staff spaces)
  width:       number;
  height:      number;
  measures:    EngravingMeasure[];
  bracketElements: EngravingElement[];
  staffLines:  EngravingStaffLine[];
};

export type EngravingStaffLine = {
  staffId: string;
  x: number; y: number;
  width: number;
  lineCount: number;
};

export type EngravingMeasure = {
  measureId:  string;
  x: number; y: number;
  width:      number;
  elements:   EngravingElement[];
  spanners:   EngravingSpanner[];
};

// An EngravingElement is the atomic unit consumed by the renderer.
export type EngravingElement = {
  id:        string;
  sourceId?: string;   // ID of the ScoreEvent this was derived from
  type:      EngravingElementType;
  x: number; y: number;   // in staff spaces, relative to parent measure
  bbox:      BoundingBox;
  glyph?:    SmuflGlyph;    // if rendered as a SMuFL glyph
  path?:     PathCommand[]; // if rendered as a vector path (stems, barlines)
  children?: EngravingElement[];
};

export type EngravingElementType =
  | "notehead" | "stem" | "flag" | "beam"
  | "accidental" | "articulation" | "rest"
  | "clef" | "timesig" | "keysig" | "barline"
  | "dynamic" | "text" | "rehearsal" | "lyric"
  | "ledger" | "brace" | "bracket";

export type EngravingSpanner = {
  id:       string;
  sourceId: string;   // Spanner.id
  type:     "slur" | "tie" | "hairpin" | "ottava" | "volta" | "pedal";
  path:     PathCommand[];
  bbox:     BoundingBox;
};

export type BoundingBox = {
  left: number; top: number; right: number; bottom: number;
};

export type SmuflGlyph = {
  codepoint: number;  // Unicode codepoint in the SMuFL range
```

```
    scale?:    number;  // relative to staff-space size (default 1.0)
  };

  export type PathCommand =
    | { type: "M"; x: number; y: number }
    | { type: "L"; x: number; y: number }
    | { type: "C"; x1: number; y1: number; x2: number; y2: number; x: number; y:
number }
    | { type: "Z" };

  export type LayoutParameters = {
    pageWidth:      number;   // in staff spaces
    pageHeight:     number;
    marginTop:      number;
    marginBottom:   number;
    marginLeft:     number;
    marginRight:    number;
    staffSpacing:   number;   // between staves of different parts
    systemSpacing:  number;   // between systems
    spatium:        number;   // staff space size in pixels (for display only)
    minSystemFill:  number;   // 0.0–1.0: minimum fill before line break
    stretchFactor:  number;   // global spacing stretch
  };
```

3.3 Sub-Phase 3A: Beam Group Resolution

Beam groups determine which consecutive eighth notes (or shorter) are visually connected by beams. The rules are governed by the time signature and any explicit beam markup in the score.

```
  // File: src/core/engraving/beaming/beamResolver.ts

  // A ResolvedBeamGroup is a group of notes to be drawn with shared beams.
  export type ResolvedBeamGroup = {
    noteIds:    string[];  // ordered
    beamCounts: number[];  // how many beams connect each pair of adjacent notes
    stemDirection: "up" | "down";
  };

  // Rules:
  // 1. Notes shorter than a quarter can be beamed.
  // 2. Beaming breaks at rests.
  // 3. Beaming breaks at barlines.
  // 4. Default beam groupings follow the time signature:
  //    4/4 → beam in groups of 4 eighths (or 8 sixteenths)
  //    3/4 → beam each beat separately
  //    6/8 → beam in groups of 3 eighths
  //    Compound meters: beam by dotted beat unit.
  // 5. Explicit BeamGroup objects in the score override defaults.

  export function resolveBeamGroups(
    score: Score,
    staffId: string,
    voiceId: number,
  ): ResolvedBeamGroup[] { ... }

  // Beam count between two adjacent beamed notes:
  // 1 = eighth beams, 2 = sixteenth beams, 3 = 32nd, etc.
  // The beam count between two notes is min(beams of left, beams of right).
  export function computeBeamCount(noteType: NoteType): number {
```

```
    return { "eighth": 1, "16th": 2, "32nd": 3, "64th": 4, "128th": 5 }[noteType]
?? 0;
  }
```

3.4 Sub-Phase 3B: Stem Direction & Length

```
  // File: src/core/engraving/stems/stemResolver.ts

  // Rules for stem direction:
  // 1. Voice 1 always UP, Voice 2 always DOWN (when multiple voices on a staff).
  // 2. Single voice: stems go in the direction away from the center of the staff.
  //    Notes above middle line → stem down. Notes below → stem up.
  //    Notes on the middle line → stem down (convention).
  // 3. For a chord: use the note farthest from center to determine direction.
  // 4. For a beam group: compute the average position and apply one direction.

  export type StemResolution = {
    noteId:    string;
    direction: "up" | "down";
    tipY:      number;   // Y position of stem tip (in staff spaces from top)
    baseY:     number;   // Y position of stem base (at notehead)
  };

  // Standard stem length is 3.5 staff spaces from notehead to tip.
  // Beamed groups: stem length is adjusted so the beam is horizontal or
  // nearly horizontal. The Gould "Behind Bars" rule: beam slope is at most
  // ±2 staff spaces over the length of the group.

  export function resolveStem(
    noteId: string,
    staffPosition: number,    // 0 = bottom line
    voiceCount: number,       // 1 = single voice, 2+ = multi-voice
    voiceId: number,
    beamGroup?: ResolvedBeamGroup,
  ): StemResolution { ... }
```

3.5 Sub-Phase 3C: Accidental Placement

```
  // File: src/core/engraving/accidentals/accidentalPlacer.ts

  // Accidentals must not collide with each other or with noteheads.
  // Algorithm:
  //   1. Collect all accidentals in a chord (sorted by pitch, top to bottom).
  //   2. For each accidental, try placing it at X = notehead_x - accidental_width
- gap.
  //   3. If it collides with a previously placed accidental, shift it further
left.
  //   4. Stagger: if two accidentals are close in pitch, the left one is placed
first.
  //   Reference: Gould "Behind Bars" Chapter 5 — Accidentals.

  export type AccidentalPlacement = {
    noteId:     string;
    accidental: Accidental;
    xOffset:    number;   // negative offset from notehead center
    glyph:      SmuflGlyph;
```

```
};

export function placeAccidentals(
  notes: NoteEvent[],           // all notes at a single tick/staff
  activeKey: KeySignature,       // to determine if accidental is courtesy
): AccidentalPlacement[] { ... }
```

3.6 Sub-Phase 3D: Spring-Based Rhythmic Spacing

This is the most algorithmically complex part of the engraving engine. The horizontal spacing of notes is determined by a constraint-solving system.

```
// File: src/core/engraving/spacing/spacingSolver.ts

// A spring constraint: the slice at tick T wants at least minWidth space,
// and ideally proportionalWidth space (from durationToSpacingWidth).
export type SliceSpring = {
  tick:              number;
  minWidth:          number;   // hard minimum
  proportionalWidth: number;   // preferred width (from duration)
  extraWidth:        number;   // from accidentals, articulations, lyrics
};

// Given a set of springs and a total available width,
// solve for the X position of each slice.
//   1. Sum all minimum widths. If > available width: use minimums only.
//   2. Distribute remaining space proportionally to proportionalWidth.
//   3. Result: x[i] = sum of allocated widths for slices 0..i-1.

export function solveSpacing(
  springs:        SliceSpring[],
  availableWidth: number,
): number[] { // returns x position for each spring }

// Build springs from slice map and score events.
// This accounts for: duration, accidentals, articulations, lyrics.
export function buildSprings(
  slices:  TemporalSlice[],
  score:   Score,
  tpq?:    number,
): SliceSpring[] { ... }
```

3.7 Sub-Phase 3E: Skyline Collision System

Every system maintains a top skyline and a bottom skyline — two arrays representing the outermost extent of all placed objects. New objects are pushed outward to avoid collision.

```
// File: src/core/engraving/collision/skyline.ts

// A skyline is an array of (x, height) segments covering the system width.
export type Skyline = {
  segments: SkylineSegment[];
  direction: "up" | "down";
};

export type SkylineSegment = {
```

```
    xStart: number;
    xEnd:   number;
    y:      number;   // extremal Y (min for top skyline, max for bottom)
  };

  // Query: what is the sky height over a given X range?
  export function querySkyline(skyline: Skyline, xStart: number, xEnd: number):
number { ... }

  // Update: push the skyline out to accommodate a new object.
  export function updateSkyline(
    skyline: Skyline,
    xStart: number, xEnd: number, y: number
  ): Skyline { ... }

  // Place an object: returns the Y coordinate that avoids collision,
  // then updates the skyline.
  export function placeAboveSkyline(
    skyline: Skyline,
    xStart: number, xEnd: number,
    objectHeight: number,
    minMargin: number,
  ): { y: number; updatedSkyline: Skyline } { ... }

  // Objects that use the skyline system:
  // - dynamics (pushed below staff)
  // - articulations (pushed above/below staff)
  // - lyrics (pushed below staff)
  // - fingerings (pushed above staff)
  // - slur control points (avoid noteheads and stems)
  // - hairpins (pushed below staff)
  // - text annotations
```

3.8 Sub-Phase 3F: System Breaking

The system breaking algorithm decides how many measures fit on each system (line). This is similar to a paragraph line-breaking algorithm (like TeX's Knuth-Plass).

```
  // File: src/core/engraving/systems/systemBreaker.ts

  // Algorithm:
  //   1. Assign a "natural width" to each measure:
    //      natural_width = sum of spacing widths of all its slices + preamble
(clef/key/time)
  //   2. Walk through measures greedily:
  //      - Add measures to current system until natural_width >
system_available_width
  //      - OR until a forced system break is encountered (systemBreak on a
measure)
  //   3. When a system is full: record the break.
  //   4. After all measures are assigned to systems, justify each system:
  //      - Stretch note spacing to fill the system width.
  //      - But do NOT stretch the last system beyond minSystemFill.

  export type SystemBreakResult = {
    systems: SystemLayout[];
  };

  export type SystemLayout = {
```

```
    measureIds:     string[];
    naturalWidth:   number;
    justifiedWidth: number;   // = available system width (after justification)
    stretchFactor:  number;   // justifiedWidth / naturalWidth
  };

  export function breakIntoSystems(
    measures:       Measure[],
    measureWidths:  Map<string, number>,  // measureId → natural width
    params:         LayoutParameters,
  ): SystemBreakResult { ... }
```

3.9 Sub-Phase 3G: Slur and Tie Geometry

```
  // File: src/core/engraving/slurs/slurPlacer.ts

  // A slur is a Bezier cubic curve. The control points are computed to:
  // 1. Start at the source notehead, end at the destination notehead.
  // 2. Arch above (or below) all noteheads and stems in between.
  // 3. Avoid colliding with other slurs.

  // Algorithm:
  // 1. Determine placement (above or below) from SlurSpan.placement or from stem
direction.
  // 2. Compute default arch height: proportional to horizontal span.
  //    arch_height = 0.5 * sqrt(span_in_staff_spaces)  (empirical)
  // 3. Compute control point X positions: 1/3 and 2/3 of span.
  // 4. Compute control point Y positions: arch_height above start/end Y.
  // 5. Run collision check against skyline: if arch hits a stem, increase
arch_height.
  // 6. If SlurSpan has explicit bezier override, use those control points.

  export function computeSlurGeometry(
    span:       SlurSpan | TieSpan,
    startPos:   { x: number; y: number },
    endPos:     { x: number; y: number },
    skylineTop: Skyline,
    skylineBot: Skyline,
  ): PathCommand[] { ... }
```

3.10 The Main Engraving Entry Point

```
  // File: src/core/engraving/engrave.ts

  export function engrave(
    score:  Score,
    params: LayoutParameters,
    fonts:  FontMetrics,   // from Phase 4
  ): EngravingResult {
    // 1. Build temporal map
    const sliceMap = buildSliceMap(score);

    // 2. Resolve beam groups per staff/voice
    // 3. Resolve stem directions
    // 4. Place accidentals
    // 5. Build springs per measure
```

```
    // 6. Break into systems
    // 7. Justify systems
    // 8. Compute element positions
    // 9. Run skyline collision
    // 10. Compute slur/tie paths
    // 11. Assemble EngravingResult

    return result;
  }

  // Incremental re-engraving: only re-compute affected measures.
  // This is an optimization for interactive editing (Phase 6).
  export function reengraveFromMeasure(
    score:        Score,
    params:       LayoutParameters,
    fonts:        FontMetrics,
    fromMeasure:  number,    // re-engrave from this measure onward
    previous:     EngravingResult,
  ): EngravingResult { ... }
```

3.11 Phase 3 Deliverables

- All engraving sub-systems implemented and independently unit-tested.
- engrave() produces a valid EngravingResult for the test score from Phase 1.
- Beam grouping tests: 4/4 eighths beam in groups of 4, 6/8 beams in groups of 3.
- Stem direction tests: voice 1 up, voice 2 down, single voice follows pitch.
- Spacing tests: dotted quarter note wider than plain quarter note.
- Skyline test: dynamic placed below staff does not collide with lyric text.
- System break test: 4-measure score fits on 2 systems with correct measure distribution.

Phase 4 — Font System (Leland OTF / SMuFL)

OpenNotation uses Leland.otf from MuseScore for all musical glyphs. Leland is SMuFL-compliant, open-source (SIL Open Font License), and covers all standard music notation symbols. We do NOT generate any graphics manually — every symbol is a Unicode character rendered in Leland.

4.1 What Is SMuFL?

SMuFL (Standard Music Font Layout) is a specification for mapping music symbols to Unicode code points in the range U+E000 to U+F8FF (the Private Use Area). Every SMuFL-compliant font, including Leland, uses the same codepoints for the same symbols.

This means: if you know the SMuFL name of a glyph, you know its codepoint, and you can render it in Leland or any other SMuFL font interchangeably.

4.2 The SMuFL Glyph Map

```
// File: src/core/fonts/smufl.ts

// A subset of the SMuFL glyph map covering all glyphs used by OpenNotation.
// Full specification at: https://w3c.github.io/smufl/latest/

export const SMUFL = {
  // Noteheads
  noteheadBlack:           0xE0A4,
  noteheadHalf:            0xE0A3,
  noteheadWhole:           0xE0A2,
  noteheadDoubleWhole:     0xE0A0,
  noteheadXBlack:          0xE0A9,   // percussion X notehead

  // Rests
  restWhole:               0xE4E3,
  restHalf:                0xE4E4,
  restQuarter:             0xE4E5,
  rest8th:                 0xE4E6,
  rest16th:                0xE4E7,
  rest32nd:                0xE4E8,
  rest64th:                0xE4E9,

  // Flags (stem flags for unbeamed short notes)
  flag8thUp:               0xE240,
  flag8thDown:             0xE241,
  flag16thUp:              0xE242,
  flag16thDown:            0xE243,
  flag32ndUp:              0xE244,
  flag32ndDown:            0xE245,

  // Accidentals
  accidentalSharp:         0xE262,
  accidentalFlat:          0xE260,
  accidentalNatural:       0xE261,
  accidentalDoubleSharp:   0xE263,
  accidentalDoubleFlat:    0xE264,

  // Clefs
```

```
    gClef:                   0xE050,
    fClef:                   0xE062,
    cClef:                   0xE05C,
    gClef8vb:                0xE052,
    unpitchedPercussionClef: 0xE069,

    // Time signatures (individual digits)
    timeSig0: 0xE080, timeSig1: 0xE081, timeSig2: 0xE082,
    timeSig3: 0xE083, timeSig4: 0xE084, timeSig5: 0xE085,
    timeSig6: 0xE086, timeSig7: 0xE087, timeSig8: 0xE088,
    timeSig9: 0xE089, timeSigCommon: 0xE08A, timeSigCut: 0xE08B,

    // Articulations
    articStaccatoAbove:      0xE4A2,
    articTenutoAbove:        0xE4A4,
    articAccentAbove:        0xE4A0,
    articMarcatoAbove:       0xE4AC,
    articStaccatissimoAbove: 0xE4A6,

    // Dynamics
    dynamicPP:               0xE52B,
    dynamicP:                0xE520,
    dynamicMP:               0xE52C,
    dynamicMF:               0xE52D,
    dynamicF:                0xE522,
    dynamicFF:               0xE52F,
    dynamicSForzando:        0xE526,

    // Augmentation dot
    augmentationDot:         0xE1E7,

    // Barlines
    barlineSingle:           0xE030,
    barlineDouble:           0xE031,
    barlineFinal:            0xE032,
    repeatDot:               0xE044,

    // Brace / bracket
    brace:                   0xE000,

    // Tremolo
    tremolo1:                0xE220,
    tremolo2:                0xE221,
    tremolo3:                0xE222,

    // Ornaments
    ornamentTrill:           0xE566,
    ornamentTurn:            0xE567,
    ornamentMordent:         0xE56C,

  } as const;

  export type SmuflName = keyof typeof SMUFL;

  // Helper: get the character string for a glyph
  export function glyphChar(name: SmuflName): string {
    return String.fromCodePoint(SMUFL[name]);
  }
```

4.3 Font Metrics

The engraving engine needs to know the bounding box of each glyph to perform collision detection and spacing. These metrics come from the Leland font file and are pre-extracted into a JSON file at build time.

```
// File: src/core/fonts/metrics.ts

// GlyphMetrics are in em units (1.0 = 1 em = the font's em square).
// To convert to staff spaces: multiply by (staffSpaceSize / emSize).
export type GlyphMetrics = {
  bBoxNE: { x: number; y: number };   // bounding box north-east
  bBoxSW: { x: number; y: number };   // bounding box south-west
  stemUpSE: { x: number; y: number }; // where stem attaches (up, south-east)
  stemDownNW: { x: number; y: number };
  noteheadOrigin?: { x: number; y: number };
};

export type FontMetrics = {
  glyphs:   Record<string, GlyphMetrics>;  // keyed by SMuFL name
  engravingDefaults: EngravingDefaults;
};

// EngravingDefaults come from the Leland metadata JSON.
// They define standard widths and spacings in staff spaces.
export type EngravingDefaults = {
  staffLineThickness:     number;
  stemThickness:          number;
  beamThickness:          number;
  beamSpacing:            number;
  legerLineThickness:     number;
  legerLineExtension:     number;
  slurEndpointThickness:  number;
  slurMidpointThickness:  number;
  thinBarlineThickness:   number;
  thickBarlineThickness:  number;
  barlineSeparation:      number;
  dotDotDistance:         number;
  tupletBracketThickness: number;
};

// Runtime font loader
export type FontLoader = {
  load(fontPath: string): Promise<FontMetrics>;
  isLoaded(): boolean;
  getMetrics(): FontMetrics;
};

export function createFontLoader(): FontLoader { ... }
```

4.4 Font Loader Implementation

The font loader has two responsibilities: (1) make the Leland OTF available to the rendering context, and (2) load the SMuFL metadata JSON that contains glyph bounding boxes.

```
// File: src/core/fonts/lelandLoader.ts

// The Leland OTF is loaded as a FontFace in browser contexts.
// In Node.js contexts (tests, CLI), it is loaded via canvas or skipped.
```

```
export async function loadLeland(fontPath: string): Promise<FontMetrics> {
  // Step 1: Register the font with the browser FontFace API.
  const face = new FontFace("Leland", `url(${fontPath})`);
  await face.load();
  document.fonts.add(face);

  // Step 2: Load the SMuFL metadata JSON.
  // This JSON is extracted from the font at build time using a script
  // that reads the GLYPH_DATA table from the OTF file.
  const metadataPath = fontPath.replace(".otf", ".metadata.json");
  const response = await fetch(metadataPath);
  const raw = await response.json();

  // Step 3: Parse and normalize metrics.
  return parseMetadata(raw);
}

// The metadata JSON follows the SMuFL "glyphsWithAnchors" format:
// {
//   "noteheadBlack": {
//     "stemUpSE":  [0.132, 0.168],
//     "stemDownNW": [-0.132, -0.3],
//     "bBoxNE":    [0.932, 0.636],
//     "bBoxSW":    [-0.084, -0.648]
//   },
//   ...
// }
```

4.5 Font Adapter Interface

The font system is modular. The default adapter uses Leland. An alternative adapter could use Bravura, Petaluma, or a custom font. All adapters implement IFontAdapter.

```
// File: src/core/fonts/IFontAdapter.ts

export interface IFontAdapter {
  // Return the CSS font-family string for use in canvas.font
  readonly fontFamily: string;

  // Return the Unicode character for a given SMuFL name
  getGlyphChar(name: SmuflName): string;

  // Return bounding box metrics for a glyph
  getGlyphMetrics(name: SmuflName): GlyphMetrics;

  // Return standard engraving defaults
  readonly engravingDefaults: EngravingDefaults;

  // Is the font loaded and ready?
  readonly isReady: boolean;
}
```

4.6 Phase 4 Deliverables

- SMUFL codepoint map defined for all glyphs used by the engine.

- Leland.otf and Leland.metadata.json placed in fonts/ directory.
- FontLoader implemented: loads OTF + metadata, returns FontMetrics.
- IFontAdapter interface defined.
- LelandFontAdapter implements IFontAdapter.
- Unit tests: glyphChar("noteheadBlack") returns the correct Unicode character.
- Unit tests: getGlyphMetrics("noteheadBlack") returns correct bounding box dimensions.

Phase 5 — Rendering Pipeline

The renderer consumes the EngravingResult from Phase 3 and draws it to screen. It has two implementations: a WebGPU backend for GPU-accelerated rendering and a Canvas 2D software fallback. The renderer knows nothing about music — it only knows how to draw glyphs, paths, and text.

5.1 Renderer Interface

```
  // File: src/core/rendering/IRenderer.ts

  export interface IRenderer {
    // Initialize the renderer. Called once.
    init(canvas: HTMLCanvasElement, fonts: IFontAdapter): Promise<void>;

    // Render a complete EngravingResult. Replaces the previous frame.
    render(result: EngravingResult, viewport: Viewport): void;

    // Partial re-render: update specific pages only.
    renderPages(pageIndices: number[], result: EngravingResult, viewport:
Viewport): void;

    // Resize the canvas.
    resize(width: number, height: number): void;

    // Hit testing: given screen coords, return the EngravingElement at that point.
    hitTest(screenX: number, screenY: number): EngravingElement | null;

    // Highlight (e.g. selection): draw a highlight rect over specific element IDs.
    setHighlights(elementIds: string[]): void;

    // Dispose GPU resources.
    dispose(): void;
  }

  export type Viewport = {
    pageIndex:  number;
    scrollY:    number;   // vertical scroll in pixels
    scale:      number;   // zoom: 1.0 = 100%
    spatium:    number;   // pixels per staff space at current zoom
  };
```

5.2 Canvas 2D Renderer (Primary Implementation)

Build the Canvas 2D renderer first. It is simpler, has zero GPU setup overhead, and runs everywhere. WebGPU is added later as an optimization.

```
  // File: src/core/rendering/canvas2d/Canvas2DRenderer.ts

  export class Canvas2DRenderer implements IRenderer {
    private ctx!: CanvasRenderingContext2D;
    private fonts!: IFontAdapter;
    private lastResult?: EngravingResult;
```

```
    private elementMap = new Map<string, { element: EngravingElement; screenBBox:
BoundingBox }>();

    async init(canvas: HTMLCanvasElement, fonts: IFontAdapter): Promise<void> {
      this.ctx = canvas.getContext("2d")!;
      this.fonts = fonts;
    }

    render(result: EngravingResult, viewport: Viewport): void {
      this.lastResult = result;
      this.elementMap.clear();
      const { ctx } = this;
      const sp = viewport.spatium;   // pixels per staff space

      ctx.clearRect(0, 0, ctx.canvas.width, ctx.canvas.height);
      ctx.save();
      ctx.translate(0, -viewport.scrollY);
      ctx.scale(viewport.scale, viewport.scale);

      for (const page of result.pages) {
        this.renderPage(page, sp);
      }

      ctx.restore();
    }

    private renderPage(page: EngravingPage, sp: number): void {
      // 1. Draw page background (white rectangle)
      // 2. Draw page elements (title, composer)
      // 3. Draw each system
      for (const system of page.systems) {
        this.renderSystem(system, sp);
      }
    }

    private renderSystem(system: EngravingSystem, sp: number): void {
      // 1. Draw staff lines
      for (const line of system.staffLines) {
        this.drawStaffLines(line, sp);
      }
      // 2. Draw bracket/brace elements
      // 3. Draw each measure
      for (const measure of system.measures) {
        this.renderMeasure(measure, sp, system.x, system.y);
      }
    }

    private renderMeasure(measure: EngravingMeasure, sp: number, sysX: number,
sysY: number): void {
      for (const el of measure.elements) {
        this.renderElement(el, sp, sysX + measure.x, sysY + measure.y);
      }
      for (const spanner of measure.spanners) {
        this.renderSpanner(spanner, sp, sysX + measure.x, sysY + measure.y);
      }
    }

    private renderElement(el: EngravingElement, sp: number, baseX: number, baseY:
number): void {
      const screenX = (baseX + el.x) * sp;
      const screenY = (baseY + el.y) * sp;
```

```
      // Store for hit testing
      this.elementMap.set(el.id, {
        element: el,
        screenBBox: {
          left:   screenX + el.bbox.left * sp,
          top:    screenY + el.bbox.top * sp,
          right:  screenX + el.bbox.right * sp,
          bottom: screenY + el.bbox.bottom * sp,
        },
      });

      if (el.glyph) {
        this.drawGlyph(el.glyph, screenX, screenY, sp);
      } else if (el.path) {
        this.drawPath(el.path, screenX, screenY, sp);
      }

      if (el.children) {
        for (const child of el.children) {
          this.renderElement(child, sp, baseX + el.x, baseY + el.y);
        }
      }
    }

    private drawGlyph(glyph: SmuflGlyph, x: number, y: number, sp: number): void {
      const { ctx, fonts } = this;
      // Font size: SMuFL glyphs are designed at 1em = 4 staff spaces.
      // So fontSize = 4 * sp.
      const fontSize = 4 * sp * (glyph.scale ?? 1);
      ctx.font = `${fontSize}px "${fonts.fontFamily}"`;
      ctx.fillStyle = "#000000";
      ctx.textBaseline = "alphabetic";
      const char = String.fromCodePoint(glyph.codepoint);
      ctx.fillText(char, x, y);
    }

    private drawPath(path: PathCommand[], baseX: number, baseY: number, sp:
number): void {
      const { ctx } = this;
      ctx.beginPath();
      for (const cmd of path) {
        switch (cmd.type) {
          case "M": ctx.moveTo(baseX + cmd.x * sp, baseY + cmd.y * sp); break;
          case "L": ctx.lineTo(baseX + cmd.x * sp, baseY + cmd.y * sp); break;
          case "C": ctx.bezierCurveTo(
            baseX + cmd.x1 * sp, baseY + cmd.y1 * sp,
            baseX + cmd.x2 * sp, baseY + cmd.y2 * sp,
            baseX + cmd.x  * sp, baseY + cmd.y  * sp); break;
          case "Z": ctx.closePath(); break;
        }
      }
      ctx.fillStyle = "#000000";
      ctx.fill();
    }

    private drawStaffLines(line: EngravingStaffLine, sp: number): void {
      const { ctx } = this;
      ctx.lineWidth = 0.13 * sp;   // from EngravingDefaults.staffLineThickness
      ctx.strokeStyle = "#000000";
      for (let i = 0; i < line.lineCount; i++) {
        const y = (line.y + i) * sp;  // each staff line is 1 staff space apart
        ctx.beginPath();
```

```
        ctx.moveTo(line.x * sp, y);
        ctx.lineTo((line.x + line.width) * sp, y);
        ctx.stroke();
      }
    }

    hitTest(screenX: number, screenY: number): EngravingElement | null {
      for (const { element, screenBBox } of this.elementMap.values()) {
        if (screenX >= screenBBox.left && screenX <= screenBBox.right &&
            screenY >= screenBBox.top  && screenY <= screenBBox.bottom) {
          return element;
        }
      }
      return null;
    }

    setHighlights(elementIds: string[]): void {
      // Redraw after setting highlight state
      // Highlighted elements are drawn with a blue tint overlay
    }

    resize(width: number, height: number): void {
      this.ctx.canvas.width  = width;
      this.ctx.canvas.height = height;
      if (this.lastResult) this.render(this.lastResult, /* last viewport */ {} as
Viewport);
    }

    dispose(): void { /* no-op for Canvas 2D */ }
  }
```

5.3 WebGPU Renderer (Advanced / Phase 10)

The WebGPU renderer is built after the Canvas 2D renderer works correctly. It provides hardware-accelerated rendering for large scores and is identical from the outside (same IRenderer interface).

Key WebGPU techniques:

- Glyph atlas: pre-render all used SMuFL glyphs to a GPU texture atlas at startup.
- Instanced rendering: render all noteheads in a single draw call using GPU instancing.
- Path batching: collect all stems, barlines, and beams into a single vertex buffer.
- Page caching: render each page to a GPU texture and composite at scroll time.

5.4 Phase 5 Deliverables

- Canvas2DRenderer implements IRenderer.
- Renders a complete test score: staff lines, noteheads, stems, barlines, clef, key signature, time signature.
- Hit testing correctly identifies clicked elements by screen position.
- setHighlights() draws a selection rect around highlighted elements.
- Viewport scrolling and zoom work correctly.
- IRenderer interface defined for future WebGPU adapter.

Phase 6 — Interaction System (Editor & State Machine)

The interaction system is a formal state machine. Every user action passes through a pipeline that transforms it into a score mutation command. The command is then applied, the score is re-engraved, and the renderer updates. No shortcuts. No direct DOM mutations.

6.1 The Interaction Pipeline

```
Raw Browser Event (mousedown, keydown, etc.)
    │
    ▼
InputController         — normalizes browser events, applies DPI scaling
    │
    ▼
InteractionStateMachine — routes event based on current mode
    │
    ▼
NotationCommand         — encapsulates the mutation as an undoable object
    │
    ▼
CommandHistory          — execute() + push to undo stack
    │
    ▼
Score (mutated)
    │
    ▼
Engraving engine (reflow from affected measure)
    │
    ▼
Renderer (update changed pages)
```

6.2 Editor Modes (State Machine States)

Mode	Description
`Normal`	Default state. Click to select. Arrow keys navigate selection. Delete removes selected element.
`NoteEntry`	N key activates. Duration selected on keyboard. Click or pitch key inserts note. Tab advances.
`TextEdit`	Double-click on text element. Keys type characters. Escape exits.
`Selection`	Shift+click or click+drag creates a range selection. Ctrl+A selects all.
`Playback`	Space bar toggles playback. Transport shortcuts active.
`Pan`	Middle mouse button held: dragging scrolls the view.
`Zoom`	Ctrl+wheel or trackpad pinch changes zoom level.

6.3 The Command System (Undo/Redo)

```
// File: src/core/editor/commands/Command.ts

export interface NotationCommand {
  readonly description: string;  // e.g. "Insert note C4 quarter"
  execute(score: Score): Score;  // returns new score (immutable update)
  undo(score: Score):    Score;  // returns previous score state
}

// File: src/core/editor/commands/CommandHistory.ts
export class CommandHistory {
  private stack:   NotationCommand[] = [];
  private pointer: number = -1;

  execute(command: NotationCommand, score: Score): Score {
    // Truncate redo stack
    this.stack = this.stack.slice(0, this.pointer + 1);
    const newScore = command.execute(score);
    this.stack.push(command);
    this.pointer++;
    return newScore;
  }

  undo(score: Score): Score | null {
    if (this.pointer < 0) return null;
    const command = this.stack[this.pointer]!;
    this.pointer--;
    return command.undo(score);
  }

  redo(score: Score): Score | null {
    if (this.pointer >= this.stack.length - 1) return null;
    this.pointer++;
    const command = this.stack[this.pointer]!;
    return command.execute(score);
  }

  get canUndo(): boolean { return this.pointer >= 0; }
  get canRedo(): boolean { return this.pointer < this.stack.length - 1; }
}
```

6.4 Built-in Commands

InsertNoteCommand

```
// File: src/core/editor/commands/InsertNoteCommand.ts

export class InsertNoteCommand implements NotationCommand {
  readonly description = "Insert note";

  constructor(
    private readonly opts: {
      staffId: string;
      voiceId: number;
      tick:    number;
      pitch:   Pitch;
      duration: Duration;
    }
```

```
    ) {}

    execute(score: Score): Score {
      const note = createNoteEvent(this.opts);
      // Insert note into score.events
      // Update score.eventIndex
      // If there was a rest at this tick/duration, split or remove it
      // Re-beam the affected voice
      return updatedScore;
    }

    undo(score: Score): Score {
      // Remove the note that was inserted
      // Restore any rest that was removed
      return restoredScore;
    }
  }
```

Every mutation to the score goes through a command. Required commands:

- InsertNoteCommand — insert a note at a tick
- DeleteNoteCommand — delete a selected note, fill with rest
- ChangePitchCommand — alter pitch of existing note
- ChangeDurationCommand — alter duration of existing note
- InsertRestCommand — insert a rest
- AddArticulationCommand — add staccato, accent, etc.
- AddDynamicCommand — insert a dynamic marking
- AddSlurCommand — create a slur spanner
- AddHairpinCommand — create a crescendo/decrescendo
- InsertMeasureCommand — insert a new measure
- DeleteMeasureCommand — remove a measure
- ChangeKeySignatureCommand
- ChangeTimeSignatureCommand
- ChangeTempoCommand
- MoveElementCommand — stores the user offset (UO in the layout formula)

6.5 Selection Model

```
  // File: src/core/editor/selection/Selection.ts

  export type SelectionType = "none" | "single" | "range" | "voice" | "measure" |
"all";

  export type Selection =
    | { type: "none" }
    | { type: "single"; elementId: string; staffId: string; tick: number }
    | { type: "range"; startTick: number; endTick: number; staffId: string;
voiceId?: number }
    | { type: "measure"; measureIds: string[] }
    | { type: "all" };

  // The selection is part of editor state, NOT the score model.
```

```
// It lives in the EditorState.

export type EditorState = {
  score:          Score;
  selection:      Selection;
  mode:           EditorMode;
  noteEntryState: NoteEntryState | null;
  playbackState:  PlaybackState | null;
  viewport:       Viewport;
  history:        CommandHistory;
};
```

6.6 Note Entry State

```
// File: src/core/editor/noteentry/NoteEntryState.ts

export type NoteEntryState = {
  // Current cursor position
  staffId:  string;
  voiceId:  number;
  tick:     number;

  // Current duration selection (changed by number keys 1-9)
  duration: Duration;

  // Accidental override (changed by up/down arrow in note entry)
  accidentalOverride: Accidental | null;

  // Chord mode: if true, next pitch adds to existing chord rather than advancing
  chordMode: boolean;

  // Tie mode: next note will be tied to current note
  tieNext: boolean;
};

// Key bindings in NoteEntry mode:
// 1 = 64th note, 2 = 32nd, 3 = 16th, 4 = eighth, 5 = quarter,
// 6 = half, 7 = whole, 8 = breve
// . = toggle dot
// A-G = insert note at that pitch (nearest to current cursor position)
// Up/Down = alter last inserted note by semitone
// Ctrl+Up/Down = alter by octave
// R = insert rest of current duration
// T = tie to next note
// Escape = exit note entry
// Delete/Backspace = delete note and back up cursor
```

6.7 Mouse-to-Pitch Translation

When the user clicks on a staff in note entry mode, the click must be translated to a pitch. This is the quantization that makes notation feel snappy.

```
// File: src/core/editor/input/pitchFromMouse.ts

export function pitchFromMousePosition(
  screenX: number,
```

```
    screenY: number,
    hitElement: EngravingElement | null,
    system: EngravingSystem,
    staffId: string,
    clef: ClefType,
    viewport: Viewport,
  ): { pitch: Pitch; tick: number } {
    // Step 1: Convert screen Y to staff-space Y (relative to staff top line)
    const staffLine = system.staffLines.find(l => l.staffId === staffId)!;
    const staffTopY = staffLine.y * viewport.spatium;
    const relY = (screenY - staffTopY) / viewport.spatium;  // in staff spaces

    // Step 2: Quantize to nearest half-space (line or space)
    // Staff positions: 0=bottom line, 0.5=first space, 1=second line, etc.
    // (Staff Y is inverted: top line = 0 in staff spaces from top = position 4)
    const staffPosition = Math.round(relY * 2) / 2;

    // Step 3: Convert staff position to pitch using the active clef
    const pitch = staffPositionToPitch(staffPosition, clef);

    // Step 4: Convert screen X to the nearest valid tick
    // (Use the slice map to find the nearest slice to the click X position)
    const tick = xToNearestTick(screenX, system, viewport);

    return { pitch, tick };
  }
```

6.8 The EditorController

The EditorController is the main API surface for embedding applications. It owns the EditorState and orchestrates the pipeline from input to render.

```
// File: src/core/editor/EditorController.ts

export class EditorController {
  private state:    EditorState;
  private renderer: IRenderer;
  private fonts:    IFontAdapter;

  constructor(opts: {
    canvas:   HTMLCanvasElement;
    score:    Score;
    fonts:    IFontAdapter;
    renderer: IRenderer;
    params:   LayoutParameters;
  }) { ... }

  // Score mutation via command (the ONLY way to change the score)
  dispatch(command: NotationCommand): void {
    const newScore = this.state.history.execute(command, this.state.score);
    this.state = { ...this.state, score: newScore };
    this.reengrave();
  }

  undo(): void { ... }
  redo(): void { ... }

  // Keyboard handler: routes to state machine
  handleKeyDown(event: KeyboardEvent): void { ... }
```

```
    // Mouse handlers
    handleMouseDown(event: MouseEvent): void { ... }
    handleMouseMove(event: MouseEvent): void { ... }
    handleMouseUp(event: MouseEvent): void { ... }

    // Load a new score
    loadScore(score: Score): void { ... }

    private reengrave(): void {
      const result = engrave(this.state.score, this.layoutParams, this.fonts);
      this.renderer.render(result, this.state.viewport);
    }

    // Subscribe to state changes (for framework bindings)
    subscribe(listener: (state: EditorState) => void): () => void { ... }
  }
```

6.9 Phase 6 Deliverables

- All command classes implemented and unit tested (execute + undo roundtrip for each).
- CommandHistory: undo/redo 10 operations correctly restores each intermediate state.
- InteractionStateMachine: key events in NoteEntry mode produce correct InsertNoteCommands.
- Mouse click on staff in NoteEntry mode produces correct pitch and tick.
- Selection model: single click selects element, shift+click extends range.
- EditorController dispatches commands, re-engravs, and re-renders in under 16ms for a 4-measure score on modern hardware.

Phase 7 — Playback Engine

The playback engine traverses the semantic score graph and produces a timed sequence of note-on/note-off events. It does NOT traverse rendered objects. The score model is the playback program.

7.1 IPlaybackAdapter Interface

The playback engine generates abstract note events. The adapter translates them to actual audio. This makes OpenNotation audio-backend-agnostic.

```
// File: src/core/playback/adapters/IPlaybackAdapter.ts

export interface IPlaybackAdapter {
  // Called before playback starts. Load any audio resources needed.
  prepare(score: Score): Promise<void>;

  // Schedule a note to be played.
  scheduleNote(opts: {
    midiPitch:  number;     // 0–127
    velocity:   number;     // 0–127
    durationMs: number;
    startMs:    number;     // milliseconds from now
    channel:    number;     // MIDI channel 0–15
    program:    number;     // MIDI program 0–127
  }): void;

  // Called when playback should stop immediately.
  stop(): void;

  // Called when playback is paused.
  pause(): void;

  // Resume from paused position.
  resume(): void;

  // Current playback position in milliseconds from score start.
  readonly currentMs: number;
}

// Tone.js adapter lives in src/core/playback/adapters/ToneJsAdapter.ts
// It implements IPlaybackAdapter using Tone.Part and Tone.PolySynth.
// Users who want a different audio backend implement this interface.
```

7.2 Playback Traversal

The traversal algorithm walks the score measure by measure, resolving repeats, voltas, D.S., D.C., codas, and segnos as control-flow instructions.

```
// File: src/core/playback/traversal/PlaybackTraverser.ts

// A PlaybackEvent is an abstract note-on/note-off scheduled at a real time.
export type PlaybackEvent = {
  kind:       "note-on" | "note-off";
  midiPitch:  number;
```

```
    velocity:   number;
    atMs:       number;
    channel:    number;
    program:    number;
  };

  // Build the complete ordered list of PlaybackEvents for a score.
  // This resolves ALL repeats, voltas, and jump instructions.
  export function buildPlaybackSequence(score: Score): PlaybackEvent[] {
    // Algorithm:
    // 1. Build a linear "measure play order" that accounts for repeats:
    //    e.g. [m1, m2, m3, m2, m3, m4] for a section with one repeat.
    // 2. For each measure in play order, walk all events in all voices.
    // 3. For each NoteEvent: compute startMs from TimeMap, compute durationMs.
    // 4. Apply dynamic/velocity: ppp=16, pp=32, p=48, mp=64, mf=80, f=96, ff=112,
fff=127.
    // 5. Emit note-on at startMs, note-off at startMs + durationMs.
    // 6. Account for ties: tied notes do not get note-off until the last note in
the chain.
    // 7. Account for staccato: shorten note-off to 50% of duration.
    // 8. Account for tenuto: hold note-off to 100% of duration (default is 95%).
    ...
  }

  // Repeat resolution: build the ordered play list of measure IDs.
  export function resolveRepeats(score: Score): string[] {
    // Walk score.measures in order.
    // When encountering repeatEnd with repeatCount=2:
    //   jump back to matching repeatStart.
    // When encountering volta brackets:
    //   first pass plays volta 1, second pass skips volta 1 and plays volta 2.
    // D.S. al Coda, D.C. al Coda: implement as explicit jumps.
    ...
  }
```

7.3 Playback Controller

```
  // File: src/core/playback/PlaybackController.ts

  export class PlaybackController {
    private adapter:   IPlaybackAdapter;
    private sequence:  PlaybackEvent[] = [];
    private playing:   boolean = false;
    private startedAt: number = 0;

    // Called when the score changes: rebuild the sequence.
    setScore(score: Score): void {
      this.sequence = buildPlaybackSequence(score);
    }

    play(fromMs = 0): void {
      this.startedAt = performance.now() - fromMs;
      this.playing = true;
      // Schedule all events from fromMs onward.
      const future = this.sequence.filter(e => e.atMs >= fromMs);
      for (const event of future) {
        this.adapter.scheduleNote({
          midiPitch: event.midiPitch,
          velocity:  event.velocity,
```

```
          durationMs: 50, // note-on only; note-off handled separately
          startMs:   event.atMs - fromMs,
          channel:   event.channel,
          program:   event.program,
        });
      }
    }

    stop(): void { this.adapter.stop(); this.playing = false; }
    pause(): void { this.adapter.pause(); this.playing = false; }
    resume(): void { this.adapter.resume(); this.playing = true; }

    // Current score position in ticks
    get currentTick(): number {
      const ms = this.adapter.currentMs;
      // Convert ms back to ticks using TimeMap.
      return msToTick(/* score */ {} as Score, ms);
    }
  }
```

7.4 Phase 7 Deliverables

- IPlaybackAdapter interface defined.
- ToneJsAdapter implements IPlaybackAdapter (optional dep: Tone.js).
- buildPlaybackSequence() correctly orders all events with real-time timestamps.
- resolveRepeats() correctly handles: single repeat, multiple repeats, 1st/2nd volta.
- Tied notes produce a single long note-on without intermediate note-offs.
- Dynamic markings map to velocity values.
- PlaybackController.play() starts audio within 10ms of call.

Phase 8 — Serialization (MusicXML, JSON, MIDI)

The serialization module converts between the internal Score model and standard file formats. MusicXML interoperability is mandatory. The native JSON format is for fast save/load. MIDI export enables audio workflows.

8.1 MusicXML Import

```
// File: src/core/serialization/musicxml/import.ts

// MusicXML 4.0 is the target standard.
// Use the DOMParser API in browser contexts.
// Use a SAX/DOM XML parser in Node.js contexts.

export async function importMusicXML(xmlString: string): Promise<Score> {
  const doc = new DOMParser().parseFromString(xmlString, "application/xml");

  // The <score-partwise> element is the root.
  const root = doc.querySelector("score-partwise");
  if (!root) throw new Error("Not a valid partwise MusicXML document");

  // 1. Parse <part-list> → create Parts and Staves
  // 2. Parse each <part> → parse each <measure>
  //    a. Parse <attributes>: divisions, key, time, clef
  //    b. Parse <note>: pitch, duration, type, dot, tie, beam, chord
  //    c. Parse <direction>: dynamics, tempo, rehearsal marks
  //    d. Parse <barline>: repeat, volta
  // 3. Resolve ties (tie-start to tie-end across measures)
  // 4. Build beam groups from beam elements
  // 5. Build spanners from direction/wedge and slur elements

  return score;
}

// MusicXML uses "divisions" as its tick unit.
// Convert: opennotation_ticks = (musicxml_duration / divisions) * TPQ
```

8.2 MusicXML Export

```
// File: src/core/serialization/musicxml/export.ts

export function exportMusicXML(score: Score): string {
  const parts: string[] = [];

  // Build <part-list>
  // Build each <part> with its <measure> elements
  // Each measure: <attributes> (when changed), then <note> elements
  // Each note: <pitch>, <duration>, <type>, <dot>, <tie>, <notations>

  return `<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE score-partwise PUBLIC "-//Recordare//DTD MusicXML 4.0 Partwise//EN"
  "http://www.musicxml.org/dtds/partwise.dtd">
<score-partwise version="4.0">
  <part-list>...</part-list>
```

```
  ${parts.join("\n")}
</score-partwise>`;
  }
```

8.3 Native JSON Format

```
// File: src/core/serialization/json/nativeFormat.ts

// The native format is a direct serialization of the Score type.
// Maps are serialized as arrays of [key, value] pairs.

export function exportJSON(score: Score): string {
  const serializable = {
    ...score,
    events:      [...score.events.entries()],
    eventIndex:  [...score.eventIndex.entries()],
    spanners:    [...score.spanners.entries()],
    beamGroups:  [...score.beamGroups.entries()],
    tuplets:     [...score.tuplets.entries()],
  };
  return JSON.stringify(serializable, null, 2);
}

export function importJSON(json: string): Score {
  const raw = JSON.parse(json);
  return {
    ...raw,
    events:     new Map(raw.events),
    eventIndex: new Map(raw.eventIndex),
    spanners:   new Map(raw.spanners),
    beamGroups: new Map(raw.beamGroups),
    tuplets:    new Map(raw.tuplets),
  } as Score;
}
```

8.4 MIDI Export

```
// File: src/core/serialization/midi/export.ts

// Build a Type 1 MIDI file (one track per part).
// Does not handle repeats (export is linear).

export function exportMIDI(score: Score): Uint8Array {
  // 1. Create MIDI header chunk: format=1, nTracks=parts.length+1, tpq=480
  // 2. Create tempo track: place SetTempo events from TempoEvents in score
  // 3. For each part: create a MIDI track
  //    - Note On events at correct tick positions
  //    - Note Off events (tick + duration)
  //    - Program Change at tick 0 (MIDI program from Part)
  // 4. Encode as Standard MIDI File binary format
  // 5. Return as Uint8Array
  ...
}
```

8.5 Phase 8 Deliverables

- MusicXML import: successfully imports all example MusicXML files from the W3C test suite (basic subset).
- MusicXML export: round-trip (import → export → import) produces equivalent scores.
- JSON round-trip: exported score re-imports identically.
- MIDI export: exported file opens correctly in GarageBand or MuseScore.
- All serializers handle: multiple parts, multiple staves, repeats, dynamics, articulations, slurs.

Phase 9 — Platform Adapters & Public API

OpenNotation's engine core is platform-agnostic. Phase 9 creates the thin adapters that connect it to specific deployment targets, and defines the clean public API surface that consumers use.

9.1 The Public API (src/bindings/vanilla/index.ts)

```
  // This is the primary entry point for vanilla JS / non-React consumers.

  export { EditorController } from "../../core/editor/EditorController";
  export { engrave }          from "../../core/engraving/engrave";
  export { Canvas2DRenderer } from
"../../core/rendering/canvas2d/Canvas2DRenderer";
  export { createFontLoader } from "../../core/fonts/lelandLoader";
  export { importMusicXML, exportMusicXML } from
"../../core/serialization/musicxml";
  export { importJSON, exportJSON }          from "../../core/serialization/json";
  export { exportMIDI }                      from "../../core/serialization/midi";
  export { createScore, createNoteEvent }    from "../../core/model/factory";
  export type { Score, NoteEvent, Pitch, Duration } from "../../core/model";
  export type { LayoutParameters, EngravingResult }  from
"../../core/engraving/types";
  export type { IRenderer }                          from
"../../core/rendering/IRenderer";
  export type { IPlaybackAdapter }                   from
"../../core/playback/adapters";

  // High-level convenience function: create a complete editor in one call.
  export function createEditor(opts: {
    canvas:    HTMLCanvasElement;
    score?:    Score;
    fontPath?: string;
    params?:   Partial<LayoutParameters>;
  }): Promise<EditorController> {
    ...
  }
```

9.2 React Bindings

```
  // File: src/bindings/react/useOpenNotation.ts

  import { useEffect, useRef, useState, useCallback } from "react";

  export function useOpenNotation(opts: {
    score: Score;
    params?: Partial<LayoutParameters>;
  }) {
    const canvasRef    = useRef<HTMLCanvasElement>(null);
    const controllerRef = useRef<EditorController | null>(null);
    const [ready, setReady] = useState(false);

    useEffect(() => {
      if (!canvasRef.current) return;
      createEditor({ canvas: canvasRef.current, score: opts.score, params:
opts.params })
```

```
        .then(controller => {
          controllerRef.current = controller;
          setReady(true);
        });
      return () => controllerRef.current?.dispose?.();
    }, []);

    const dispatch = useCallback((cmd: NotationCommand) => {
      controllerRef.current?.dispatch(cmd);
    }, []);

    const undo = useCallback(() => controllerRef.current?.undo(), []);
    const redo = useCallback(() => controllerRef.current?.redo(), []);

    return { canvasRef, ready, dispatch, undo, redo, controller: controllerRef };
  }

  // Example consumer:
  // function MyNotation() {
  //   const score = useMemo(() => createMinimalScore(), []);
  //   const { canvasRef, dispatch } = useOpenNotation({ score });
  //   return <canvas ref={canvasRef} width={800} height={600} />;
  // }
```

9.3 Tauri Platform Adapter

In a Tauri application, file I/O (open/save score files) is handled through Tauri's IPC bridge rather than browser APIs. The platform adapter wraps this.

```
  // File: src/platform/tauri/fileAdapter.ts

  import { open, save } from "@tauri-apps/api/dialog";
  import { readTextFile, writeTextFile, writeBinaryFile } from
"@tauri-apps/api/fs";

  export async function openScoreFile(): Promise<Score | null> {
    const path = await open({ filters: [
      { name: "MusicXML", extensions: ["xml", "musicxml", "mxl"] },
      { name: "OpenNotation", extensions: ["on.json"] },
    ]});
    if (!path || Array.isArray(path)) return null;
    const content = await readTextFile(path as string);
    if (path.endsWith(".json")) return importJSON(content);
    return importMusicXML(content);
  }

  export async function saveScoreFile(score: Score): Promise<void> {
    const path = await save({ filters: [
      { name: "MusicXML", extensions: ["musicxml"] },
      { name: "OpenNotation JSON", extensions: ["on.json"] },
      { name: "MIDI", extensions: ["mid"] },
    ]});
    if (!path) return;
    if (path.endsWith(".mid")) {
      const midi = exportMIDI(score);
      await writeBinaryFile(path, midi);
    } else if (path.endsWith(".json")) {
      await writeTextFile(path, exportJSON(score));
    } else {
```

```
      await writeTextFile(path, exportMusicXML(score));
    }
  }
```

9.4 Phase 9 Deliverables

- Public vanilla API documented and exported correctly from package root.
- useOpenNotation React hook works in a minimal React app.
- Tauri file adapter opens and saves MusicXML and JSON formats.
- Electron adapter analogous to Tauri adapter, using Node.js fs module.
- Integration test: createEditor() → insertNote → exportMusicXML → importMusicXML produces equivalent score.

Phase 10 — Optimization (WebGPU, WASM, Virtualization)

Phase 10 is performance engineering. The engine must work correctly before it is optimized. These techniques unlock very large scores (symphonies, opera scores) and smooth 60fps interaction.

10.1 WebGPU Renderer

Glyph Atlas

Pre-render all used SMuFL glyphs to a 2D GPU texture atlas at startup. Each glyph is rasterized once using Canvas 2D, copied to a GPU texture, and indexed by glyph codepoint.

```
// Build atlas at startup:
// 1. Enumerate all SmuflGlyph values used in the current EngravingResult.
// 2. For each glyph, measure its bounds using ctx.measureText().
// 3. Pack glyphs into a 4096x4096 texture atlas (row packing).
// 4. Upload to GPU as a sampler2D texture.
// 5. Store UV coordinates for each glyph codepoint.

// Per frame, render all noteheads in ONE draw call:
// - Build a vertex buffer: one quad per notehead, UV from atlas.
// - Use GPU instancing: position as per-instance attribute.
// - Single draw call: drawInstancedPrimitives(quads, noteheadCount).
```

Path Batching

Collect all stems, barlines, beams, and staff lines into a single vertex buffer per frame. Use a triangle-strip representation for thick lines (avoids GL_LINE_WIDTH limitations).

Page Caching

Render each page to an offscreen GPU texture. During scrolling, composite cached textures. Only re-render pages that contain modified measures. This makes scrolling through a 100-measure score as fast as scrolling through a 4-measure score.

10.2 WASM Modules

Two hot paths are candidates for Rust/WASM optimization: the spacing solver and the skyline collision system. Both are pure computations with no DOM dependency.

```
// rust/spacing/src/lib.rs
use wasm_bindgen::prelude::*;

#[wasm_bindgen]
pub fn solve_spacing(springs: &[f64], available_width: f64) -> Vec<f64> {
  // Same algorithm as TypeScript implementation but 10-50x faster.
  // springs is a flat array: [minWidth, proportionalWidth, extraWidth, ...]
  ...
```

```
  }

  // TypeScript bridge:
  // File: src/core/wasm/spacingBridge.ts
  let wasmSolver: typeof import("../../rust/spacing/pkg") | null = null;

  export async function initWasmSpacing(): Promise<void> {
    wasmSolver = await import("../../rust/spacing/pkg");
  }

  export function solveSpacingOptimized(springs: SliceSpring[], available: number):
number[] {
    if (wasmSolver) {
      const flat = springs.flatMap(s => [s.minWidth, s.proportionalWidth,
s.extraWidth]);
      return Array.from(wasmSolver.solve_spacing(new Float64Array(flat),
available));
    }
    return solveSpacing(springs, available);  // TypeScript fallback
  }
```

10.3 Incremental Re-Engraving

When the user inserts a single note, only the affected measure and potentially adjacent measures need to be re-engraved. The system break algorithm may shift subsequent measures, but notes within unchanged measures do not need re-computation.

```
  // The reengraveFromMeasure() function (defined in Phase 3) takes a
  // "dirty measure" index and reuses the previous EngravingResult for all
  // earlier measures. This makes interactive editing sub-16ms for typical edits.

  // Implementation strategy:
  // 1. Tag each measure's EngravingMeasure with a "version hash"
  //    based on the event IDs and their hashed content.
  // 2. On re-engrave, compare version hashes.
  // 3. Reuse unchanged measure geometry verbatim.
  // 4. Only re-run the full pipeline for measures with changed hashes,
  //    plus the system-breaking step (which may shift all subsequent measures).
```

10.4 Score Virtualization

For very large scores (500+ measures), do not engrave all pages at once. Only engrave pages within a viewport window ± 2 pages. Engrave surrounding pages lazily in idle time using requestIdleCallback.

```
  export class VirtualizedEngravingManager {
    private engraved = new Map<number, EngravingPage>();  // pageIndex →
EngravingPage
    private engraveQueue: number[] = [];

    setViewport(viewport: Viewport): void {
      const currentPage = viewport.pageIndex;
      const priority = [currentPage - 1, currentPage, currentPage + 1, currentPage
+ 2];
      // Immediately engrave priority pages.
      // Queue surrounding pages for idle engraving.
```

```
    }

    private idleEngrave(): void {
      requestIdleCallback(deadline => {
        while (deadline.timeRemaining() > 2 && this.engraveQueue.length > 0) {
          const pageIndex = this.engraveQueue.shift()!;
          this.engraved.set(pageIndex, /* engrave page */ {} as EngravingPage);
        }
        if (this.engraveQueue.length > 0) this.idleEngrave();
      });
    }
  }
```

10.5 Phase 10 Deliverables

- WebGPU renderer renders a 100-measure score at 60fps with smooth scrolling.
- Glyph atlas covers all SMuFL glyphs used in the test score suite.
- WASM spacing solver: benchmarks at least 10x faster than TypeScript solver for 200+ slice measures.
- Incremental re-engraving: single note insert re-renders in <16ms on a 100-measure score.
- Virtualized engraving: a 500-measure score loads initial view in <200ms.

Appendix A — Instructions for Cursor.AI

These are the standing rules for every Cursor.AI session on this codebase. Paste them at the top of every conversation.

A.1 Architectural Rules (Never Violate)

12. The score model NEVER contains pixel coordinates. If you are about to add x, y, or screenX to any type in src/core/model/, stop and reconsider the architecture.
13. The renderer NEVER imports from src/core/model/. The only type it receives is EngravingResult.
14. Every score mutation goes through a NotationCommand. Never mutate score properties directly outside of a command's execute() or undo() method.
15. The engraving engine is a pure function. It has no side effects, does not mutate its inputs, and produces identical output for identical inputs.
16. Module boundaries are enforced. Do not add an import that crosses the allowed dependency graph defined in Phase 0.
17. The layout formula AP + LO + UO must be respected. User offsets (UO) are stored in score events. They are ADDED to the engraving result in the renderer. They are never baked into the engraving geometry.

A.2 Code Style Rules

18. All types are explicit. No implicit any. Use unknown over any when the type is truly unknown.
19. Use readonly on all array properties that should not be mutated externally.
20. Prefer type over interface for all data models (interfaces for classes/implementations).
21. Every public function has a JSDoc comment explaining what it does, its parameters, and its return value.
22. Every module has an index.ts that explicitly re-exports its public API.
23. Factory functions for all Score nodes — never raw object literals.

A.3 Testing Rules

24. Every new function gets at least one unit test before the PR is complete.
25. Engraving tests: assert on geometry values (x positions, stem directions, beam groupings).
26. Command tests: execute + undo must return a score equivalent to the original.
27. Round-trip tests: importMusicXML(exportMusicXML(score)) must produce an equivalent score.

A.4 When Adding a New Feature

- Step 1: Does this belong in the semantic model? If it is musical meaning, add it to src/core/model/.
- Step 2: Does engraving need to produce geometry for it? Update the engraving pipeline and add an EngravingElement type.
- Step 3: Does the renderer need to draw it? Add a renderElement branch in Canvas2DRenderer.
- Step 4: Does the user need to interact with it? Add a NotationCommand and update the InteractionStateMachine.
- Step 5: Does serialization need to preserve it? Update MusicXML import/export and JSON format.
- Step 6: Does playback need to handle it? Update the playback traverser.

A.5 Pitfalls to Avoid

Never Do These

• Storing absolute screen positions in the Score model.
• Calling engrave() inside a render loop without caching.
• Using floating-point equality for tick comparisons (use integer ticks always).
• Rendering directly from the Score model (always go through the EngravingResult).
• Creating a new Map or Set inside the hot path of the engraving engine.
• Accessing score.events with a string key without checking for undefined.
• Forgetting to update score.eventIndex after inserting or deleting an event.
• Using canvas.font with a size in px directly on glyph render (always compute from spatium).
• Implementing "undo" by storing snapshots of the full score (use command objects).

Appendix B — Phase Summary & Dependencies

Phase	Description & Dependencies
`Phase 0 — Setup`	Repo structure, tsconfig, module boundaries, eslint, vitest, empty barrel files. No musical logic.
`Phase 1 — Score Model`	All TypeScript types: Score, Part, Staff, Measure, NoteEvent, RestEvent, Spanner, etc. Factory + query functions. Unit tests.
`Phase 2 — Temporal`	TemporalSliceMap builder, spacing weight computation, TimeMap. Depends on Phase 1.
`Phase 3 — Engraving`	Beam resolver, stem solver, accidental placer, spacing solver, skyline collision, system breaker, slur geometry, main engrave() function. Depends on Phases 1–2.
`Phase 4 — Fonts`	SMuFL codepoint map, Leland OTF loader, FontMetrics type, IFontAdapter interface. No dependencies.
`Phase 5 — Rendering`	Canvas2DRenderer, IRenderer interface, hit testing, selection highlight. Depends on Phases 3–4.
`Phase 6 — Interaction`	NotationCommand, CommandHistory, EditorStateMachine, NoteEntryState, Selection, EditorController. Depends on Phases 1–5.
`Phase 7 — Playback`	buildPlaybackSequence, resolveRepeats, PlaybackController, IPlaybackAdapter, ToneJsAdapter. Depends on Phases 1–2.
`Phase 8 — Serialization`	MusicXML import/export, JSON round-trip, MIDI export. Depends on Phase 1 only.
`Phase 9 — Platform`	Public API, React hooks, Tauri adapter, Electron adapter. Depends on all previous phases.
`Phase 10 — Optimization`	WebGPU renderer, WASM spacing/collision, incremental engraving, score virtualization. Depends on all previous phases.

— End of OpenNotation Implementation Plan —